1. Explain the difference between frontend, backend, and full-stack development with suitable real-world examples.

ANS-**1. Difference between Frontend, Backend, and Full-Stack Development**

Frontend Development:
Frontend is the part of a website or app that users can see and use. It includes buttons, images, menus, and pages. It is mainly built using HTML, CSS, and JavaScript.

Example: On Amazon, the product page, search bar, and "Add to Cart" button are part of the frontend.

Backend Development:
Backend is the part that works in the background. It handles data, user login, and connects with the database. Users cannot see it directly.

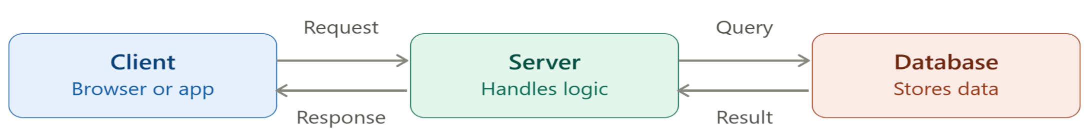
Example: When you log in to Amazon, the backend checks your username and password and shows your account details.

Full-Stack Development:
A full-stack developer works on both the frontend and backend. They can build the complete website or application.

Example: A full-stack developer can create an online shopping website with its design, login system, database, and order management.

2. Create a simple diagram showing how the client-server model works in web architecture.

ANS–



Client sends a request, server processes the logic, and database stores/returns the actual data. Response travels back the same path in reverse.

3. Describe how a browser requests and displays a web page from a web server.

ANS- When a user enters a website URL in the browser and presses Enter, the browser sends a request to the web server through the internet. The server receives the request, processes it, and sends the required files such as HTML, CSS, JavaScript, and images back to the browser. The browser reads these files, arranges the content properly, and displays the web page on the user's screen so the user can interact with it.

4. Identify and list the tools required to set up a web development environment. Explain the

purpose of each.

ANS- **4. Tools Required to Set Up a Web Development Environment**

1. **Code Editor (VS Code):** Used to write and edit HTML, CSS, JavaScript, and other programming files.
2. **Web Browser (Chrome/Firefox):** Used to run, test, and view web pages during development.
3. **Git:** A version control tool used to track code changes and manage projects.
4. **Node.js:** Provides a runtime environment to run JavaScript outside the browser and install packages.
5. **Package Manager (npm):** Used to install and manage libraries and frameworks required for the project.
6. **Live Server:** Automatically refreshes the browser whenever changes are made to the code, making development faster.

5. Explain what a web server is and give examples of commonly used servers.

ANS- **5.**

A **web server** is software that stores website files and sends them to the user's browser when a request is made. It helps users access web pages through the internet.

Common examples of web servers:

- Apache HTTP Server
- Nginx
- Microsoft IIS (Internet Information Services)
- LiteSpeed

These web servers are widely used to host websites and web applications.

6. Define the roles of a frontend developer, backend developer, and database administrator

in a project.

ANS- **Frontend Developer:**
A frontend developer creates the visible part of a website or application that users interact with. They use HTML, CSS, and JavaScript to design pages, buttons, forms, and menus. Their main goal is to make the website attractive, responsive, and easy to use.

Backend Developer:
A backend developer works on the server side of the application. They write the business logic, handle user requests, connect the application with the database, and ensure that all features work properly and securely.

Database Administrator (DBA):
A Database Administrator (DBA) manages and maintains the database of a project. They store data securely, perform backups, improve database performance, and ensure that data is organized, accurate, and available whenever required.

7. Install VS Code and configure it for HTML, CSS, and JavaScript development. Take a

screenshot of the setup.

ANS-

1. **Download and install Visual Studio Code (VS Code) from the official website.**
2. **Open VS Code and install the following extensions:**
   - **Live Server**
   - **HTML CSS Support**
   - **JavaScript (ES6) Code Snippets**
3. **Create a new folder and add three files:**
   - `index.html`
   - `style.css`
   - `script.js`
4. **Open `index.html` using Live Server to check that the setup is working.**

8. Explain the difference between static and dynamic websites. Provide an example of

each.

ANS- 9. Five Web Browsers and Their Rendering Engines

Web Browser	Rendering Engine
Google Chrome	Blink
Mozilla Firefox	Gecko
Microsoft Edge	Blink
Apple Safari	WebKit
Opera	Blink

How Rendering Engines Differ:

A rendering engine is the part of a browser that reads HTML, CSS, and JavaScript and displays a web page. Different browsers use different rendering engines, so a website may look or perform slightly differently on each browser. For example, Chrome, Edge, and Opera use Blink, Firefox uses Gecko, and Safari uses WebKit.

9. Research and list five web browsers. Explain how rendering engines differ between

them.

1. **Google Chrome – Uses the Blink rendering engine.**
2. **Mozilla Firefox – Uses the Gecko rendering engine.**
3. **Microsoft Edge – Uses the Blink rendering engine.**
4. **Apple Safari – Uses the WebKit rendering engine.**
5. **Opera – Uses the Blink rendering engine.**

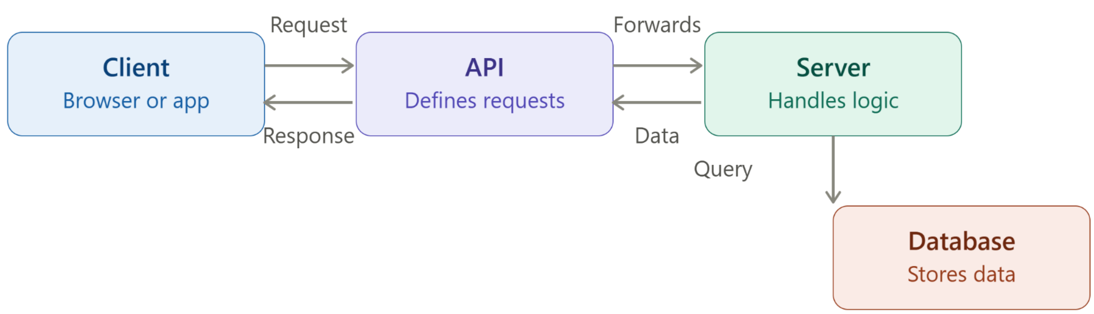
How Rendering Engines Differ:

A rendering engine is the part of a web browser that converts HTML, CSS, and JavaScript into the web page you see on the screen. Different browsers use different rendering engines, which can affect page loading speed, performance, compatibility, and the way some web pages are displayed. For example, Chrome, Edge, and Opera use Blink, Firefox uses Gecko, while Safari uses WebKit. Although most websites look similar across browsers, small differences in rendering and performance may still exist.

10. Draw a labeled diagram showing the basic web architecture flow — client, server,

database, and APIs.

ANS-



The API sits between client and server as the defined contract for requests. Server talks to the database separately, then sends the result back up the same chain.